

Aula 4: Persistência de Dados com Hibernate ORM e Quarkus Panache

Bem-vindos à nossa quarta aula! Até agora, os nossos dados (Clientes e Produtos) estavam apenas em listas temporárias no código, sendo perdidos sempre que reiniciávamos a aplicação. Hoje, vamos mudar isso e conectar nossa API a um banco de dados.

Parte 1: Hibernate ORM e Quarkus Panache

Para salvar objetos Java (como um `Cliente`) em um banco de dados relacional (em formato de tabelas e linhas), precisamos de uma "ponte" entre esses dois mundos.

- **Hibernate ORM:** É a ferramenta de Mapeamento Objeto-Relacional (ORM) mais famosa do ecossistema Java. Ele traduz automaticamente as nossas classes Java em tabelas do banco de dados e converte os nossos objetos em comandos SQL (INSERT, UPDATE, SELECT, DELETE) por trás dos panos.
- **Quarkus Panache:** Escrever código para acessar o banco de dados tradicionalmente exigia a criação de muitas classes e configurações repetitivas (como os famosos DAOs). O **Panache** é uma biblioteca criada pelo Quarkus em cima do Hibernate que simplifica tudo isso ao extremo. Ele foca na produtividade do desenvolvedor, permitindo usar padrões modernos (como o *Active Record*) onde a própria entidade ganha métodos prontos para gerenciar seus dados com comandos simples, como `Cliente.persist()` ou `Cliente.listAll()`.

Parte 2: Configurando e Integrando o Banco de Dados

Adicionando Dependências

Para começar a trabalhar com o banco de dados, precisaremos adicionar novas extensões ao nosso projeto:

- `hibernate-orm-panache` : A extensão do Panache para mapeamento de dados.
- `jdbc-h2` : O driver de conexão para o banco de dados **H2**.

Nota: O H2 é um banco de dados **em memória**. Ele é excelente para desenvolvimento, testes e para a nossa aula de hoje, pois não exige que instalemos um servidor de banco de dados real nas nossas máquinas. Os dados ficam armazenados na memória RAM enquanto a aplicação estiver rodando.

Configurando o Banco de Dados

O Quarkus é muito inteligente no modo de desenvolvimento. Ao detectar a extensão do H2, ele já configura a conexão quase toda sozinho. Nós apenas adicionaremos algumas regras no nosso arquivo `application.properties` para garantir que o banco crie ou atualize as nossas tabelas automaticamente com base no nosso código Java.

Criando a Entidade/Tabela Cliente

Vamos transformar a nossa classe `Cliente` em uma **Entidade**. Ao estender a classe `PanacheEntity` fornecida pelo Quarkus, o nosso `Cliente` herda automaticamente um campo `id` e todos os métodos necessários para interagir com o banco de dados. Usaremos também a anotação `@Entity` para avisar o sistema que essa classe representa uma tabela no banco.

Integrando a Camada de Dados com o Controller

Com a entidade pronta, voltaremos ao nosso `ClienteResource` (o Controller que fizemos na aula passada). Vamos remover a lógica de lista temporária e integrá-lo com o Panache. Agora, quando recebermos um `POST`, chamaremos diretamente a camada de banco de dados, tornando a nossa API real e funcional!

Parte 3: Desafio Prático dos Alunos

Depois de construirmos e integrarmos a persistência do `Cliente` juntos passo a passo, será a vez de vocês aplicarem o conhecimento para o escopo do nosso projeto de Compras!

O desafio prático será aplicar este mesmo processo para os **Produtos**. Vocês deverão:

1. Transformar a classe `Produto` em uma Entidade do Panache.
2. Atualizar o Endpoint de Produto criado na aula anterior.
3. Garantir que as operações de criação, listagem, atualização e exclusão de produtos estejam sendo salvas e lidas diretamente do banco de dados em memória.

Boa leitura e vamos ao código!